Project Setup

Program Overview

RFO (RentalFleetOps) is a command-line car rental fleet management system, built for AUT COMP603 (Program Design and Construction). It is a single-user Java application that manages vehicles, staff, locations and drivers through a menu-driven console interface, reading and writing its data to plain text files rather than a database.

Its core functionality covers running rentals, workshop jobs, transfers and defleets through their full lifecycle: revenue movements (checkout, check-in, and closing a rental), non-revenue movements (new vehicle intake, car transfers, workshop jobs, and defleets), querying movement and fleet details, and generating reports (vehicles out on rental, location fleet reports, new vehicle reports, and staff rental reports). The codebase is split into four layers — model, io, service, and ui — and menu options are filtered by staff role, so different roles see different functionality throughout the application.

The README.md file included in the submission explains the program, its architecture, and its known limitations more extensively.

Test Login Credentials

Staff ID	Password	Role
ONZP774	Rana1251997	MANAGER
AGNT002	agent123	AGENT
MECH003	mech123	MECHANIC
FLTO004	fleet123	FLEET_OPERATOR

Appendix A: AI Interaction Log

AI Usage Log — RFO (RentalFleetOps)

Tool used: Claude (Anthropic) **Purpose:** Design assistance for Program Design and Construction Assessment 1 — architecture planning, OOP class design, and code review.

This log records design and code decisions discussed with AI, what was proposed, and what was accepted, modified, or rejected.

---

1. Core entity design

Prompt (summary)	AI Output	Outcome
Proposed Person interface → Driver, Staff implementing it	Confirmed structure was valid	Accepted
Proposed Vehicle as interface with category subclasses	Recommended **abstract class** instead, since categories share data/fields	**Modified** — changed from interface to abstract class
Asked about adding Rentable interface for vehicles	Explained Rentable as a capability interface distinct from Vehicle's identity data	Accepted
Asked whether a Comparable interface made sense	Confirmed Comparable<Vehicle> as valid for sorting	Accepted
Added Location entity	Designed its relationship to Vehicle (currentLocation) and Movement (checkout/checkin location)	Accepted
Described staff needing multi-location access + manager-granting-access workflow	Flagged as scope risk; recommended boxing the granting workflow as a stretch feature	**Modified** — accepted the data relationship, deferred the granting workflow

2. Movement hierarchy

Prompt (summary)	AI Output	Outcome
Asked whether MovementType should be a class or enum	Initially recommended enum	Rejected — user provided more detail
Clarified movement types have genuinely different required fields/actions (new car = checkin only, defleet = checkout only, rental = price/equipment/drivers, workshop = basic)	Reversed to **abstract Movement class with subclasses** (NewCarMovement, DefleetMovement, RentalMovement, WorkshopMovement), since the variation was structural	**Accepted**
Asked about collapsing WSM/SMM/CTM into one class	Suggested collapsing unless real behavioral differences exist	Accepted, pending confirmation

3. Capability interfaces (Rentable, Serviceable)

Prompt (summary)	AI Output	Outcome
Asked if a Serviceable interface made sense for workshop/service	Initially cautioned this might duplicate a status flag	Flagged as open question
Explained odometer tracking and per-category service intervals	Confirmed this justified Serviceable as a real interface (isDueForService, markServiced) tied to odometer fields	**Accepted**
Confirmed service interval should vary per vehicle category	Recommended abstract method getServiceIntervalKm() overridden per category	Accepted

4. Staff role design (major revision)

Prompt (summary)	AI Output	Outcome
Questioned whether Staff subclasses (Manager/Agent/FleetOperator/Mechanic) were justified, since they only differed in permissions, not data	Agreed this was valid; explained subclassing is only needed if roles diverge in *data*, not just behavior	Discussion
Decided not to commit to the grant-access feature now	Proposed **StaffRole enum + concrete Staff class**	**Accepted — replaced Staff subclass hierarchy with enum-based design**

5. Vehicle category design (major revision)

Prompt (summary)	AI Output	Outcome
Questioned whether Compact/Intermediate/Standard/Premium/Special subclasses were justified	Agreed this mirrored the Staff situation; recommended a VehicleCategory enum if variation is purely numeric	Discussion
Gave real example: same category contains vehicles with different GPS/fuel combinations	Identified this as instance-level variation, not category-level; recommended hasGPS/fuelType as plain fields, category as a separate enum	**Accepted — replaced Vehicle subclass hierarchy with VehicleCategory enum + instance fields**

6. Final model design decisions

Prompt (summary)	AI Output	Outcome
Confirmed Vehicle and Staff will not be abstract classes	Movement remains the sole abstract-class hierarchy in the design	**Accepted**
Confirmed checkoutLocation/checkinLocation and odometerOut/odometerIn should be uniform across all Movement subtypes	Moved these fields up to base Movement class	Accepted
Noted RentalMovement and WorkshopMovement both need drivers, future movement types likely will too	Recommended an intermediate abstract class DrivenMovement between Movement and RentalMovement/WorkshopMovement	**Accepted — added DrivenMovement as a second inheritance level**
Noted equipment (insurance, GPS, roof rack, child seat) should be an enum	Recommended Equipment as a standalone enum	Accepted
Clarified insurance is rental-specific with dynamic pricing, scoped privately to RentalMovement	Recommended Insurance as a nested enum inside RentalMovement, cost computed via calculateInsuranceCost() using category + duration	Accepted (later reversed — see Section 14)
Noted isHybrid boolean on Vehicle was insufficient	Replaced with a FuelType enum (Diesel, Unleaded, Hybrid, PlugInHybrid, Electric)	**Accepted**
Flagged that leaving checkoutStaff/checkinStaff null for NewCarMovement/DefleetMovement risked NullPointerExceptions	Presented options: manual null-checking, Optional<Staff>, or a SYSTEM placeholder Staff record	Discussion
Decided Optional was more complexity than needed	Confirmed SYSTEM placeholder Staff approach — keeps checkoutStaff/checkinStaff always non-null	**Accepted — final decision**
Introduced CarTransferMovement (CTM) — moving a vehicle between locations, requires driver(s)	Added as a new subclass extending DrivenMovement	**Accepted**

7. Constructor and field-access strategy

Prompt (summary)	AI Output	Outcome
Raised concern that full-field constructors for every class were time-consuming	Presented options: no-arg + setters, Builder pattern, or full constructors only for small classes	Discussion — deferred until all classes existed
Asked whether Movement's fields could be protected instead of private, since subclasses need direct access	Confirmed this is a legitimate pattern for abstract base classes; protected still fully blocks outside access	**Accepted — Movement's fields protected, not private**
Requested a full pass deciding which fields belong in constructors vs. set later	Went class by class: Address/Driver/Location full-constructor; Staff/Vehicle mostly constructor with a few sensible defaults; Movement's checkout-side fields via constructor, check-in-side default and get filled via checkIn(); RentalMovement's price computed, not passed in	**Accepted — finalized constructor strategy**

8. Model layer architecture — validation lives in the service layer

Prompt (summary)	AI Output	Outcome
Clarified that model setters (e.g. switchLocation) shouldn't validate	Confirmed this is a recognized pattern; updated switchLocation to a plain,	**Accepted — confirmed as the deliberate architecture for all model**

internally — validation happens in whatever layer sits between user input and the model	unvalidated setter	**classes: simple unvalidated setters, all validation in com.rfo.service/ui**

9. Vehicle class — base rate, rental cost ownership

Prompt (summary)	AI Output	Outcome
Described that rate is a per-vehicle multiplier applied against a base daily price	Proposed two-constructor split (new vehicle vs. reconstruct-from-file); calculateRentalCost() = baseDailyRate × rate × days	Accepted
Confirmed baseDailyRate should be static but not final	Static getter/setter added to Vehicle	**Accepted**

10. Rentable interface — design flaw identified and corrected

Prompt (summary)	AI Output	Outcome
Questioned whether Rentable's rentOut/returnVehicle methods belonged to Vehicle at all, since they seemed to require multi-object orchestration (checking availability, constructing a Movement)	Identified this as orchestration, not something a single Vehicle object should own	Discussion
Clarified returnVehicle isn't uniform — it differs per Movement subtype	Confirmed Movement's abstract execute() already exists to handle this; finalized Rentable to a single method	**Accepted — Rentable reduced to isAvailable() only; rentOut()/returnVehicle() removed entirely; per-movement-type vehicle state changes live in each Movement subtype's own execute()**

11. Expected vs. actual return; rental cost calculation ownership

Prompt (summary)	AI Output	Outcome
Clarified driven movements need an expected/planned return datetime and location, separate from the actual check-in values, for upfront cost estimation	Placed expectedCheckInDT/expectedCheckInLoc on DrivenMovement, the shared parent of the three subtypes that need it	**Accepted**
Described calculateRentalCost() and getRentalEstimate()	Flagged that passing RentalMovement into a Vehicle method would create an unwanted two-way dependency; recommended both methods live on RentalMovement directly	**Accepted**

12. Vehicle class review — bug fixes and natural ordering

Prompt (summary)	AI Output	Outcome
Shared Vehicle.java for review	Found: leftover calculateRentalCost (should be on RentalMovement); getServiceInterval conflating two different concepts under one hardcoded constant; isDueForService had a side effect breaking the pure-query expectation; markServiced wasn't resetting lastServiceOdometer; compareTo always returned -1, violating Comparable's contract	Fixed
Confirmed rate as the natural ordering	Provided corrected compareTo using Double.compare(this.rate, o.rate)	**Accepted**

13. MovementStatus and two-phase check-in

Prompt (summary)	AI Output	Outcome
Added MovementStatus enum (OPEN, INCOMPLETE, then-INVOICED)	Wired into Movement: defaults OPEN, transitions to INCOMPLETE inside checkIn()	Accepted
Explained not all movements have the odometer available at the exact moment of check-in (real-world example: stop the billing clock now, get the odometer later at closing)	Recognized this as a genuine two-phase process; checkIn() takes it from OPEN to INCOMPLETE (staff/time/location only), a new closeMovement(int odoIn) takes it from INCOMPLETE to closed, setting odoIn at that point	**Accepted**

Wanted closeMovement to stay abstract on Movement for polymorphic dispatch, but was concerned the signature might be wrong once subtypes were actually built	Agreed Java requires overriding methods to match exactly; deferred — no closeMovement on Movement yet, each subtype defines its own	**Deferred, later resolved in Section 15**

14. Movement subclasses finalized

Prompt (summary)	AI Output	Outcome
Shared DrivenMovement.java for review	Caught a real bug: constructor parameter named `driver` instead of `drivers`, causing `this.drivers = drivers` to self-assign the still-null field	Fixed
Was unclear on execute()'s purpose for NewCarMovement	Clarified execute() applies an already-happened event's effect to an existing Vehicle, not creation of the Vehicle itself	Accepted
Confirmed Vehicle creation and NewCarMovement creation should both be orchestrated from the service layer, not from inside NewCarMovement	Accepted — "vehicles only created via NVM" is enforced by workflow structure, not by which class's constructor is called	**Accepted**
Shared DefleetMovement.java	Caught a real bug: execute() referenced VehicleStatus.SOLD, which didn't exist in the enum (duplicate of DEFLEETED)	Fixed — SOLD removed
Described CarTransferMovement's real vs. expected arrival distinction	Finalized: constructor sets vehicle to IN_USE at departure, execute() applies real arrival data once checkIn() is called externally	Accepted
Decided to replace live-computed isDueForService() with a stored boolean flag, recalculated only at check-in	Added serviceFlag field, split isDueForService() into a pure getter plus a separate updateServiceFlag()	**Accepted**
Asked whether closeMovement should be abstract on Movement	Reasoned a concrete shared default on DrivenMovement (override-as-needed) costs nothing and saves rewriting for movement types that behave the same	**Accepted — DrivenMovement gets concrete shared execute()/closeMovement(); MovementStatus's third value renamed INVOICED → CLOSED, since INVOICED implied billing specifically**
Worked through RentalMovement's remaining pieces: Equipment with per-instance mutable dailyRate, Insurance as a tier multiplier, RSA as a flat daily charge, day-count calculation with an hour-level grace period	Implemented getInsuranceBaseRate() bracket logic, getDaysBetween() using ChronoUnit.HOURS with the grace-period rule	Accepted
Questioned the real-world payment sequencing — price must be confirmed before closing	Clarified calculateRentalCost() is independently callable once checkInDT is set; closeMovement() takes the already-calculated price as a parameter rather than recalculating	**Accepted — model layer complete**

15. Equipment/Insurance visibility bug

Prompt (summary)	AI Output	Outcome
Noticed setInsuranceTier() takes an Insurance argument, but Insurance was nested inside RentalMovement with no explicit access modifier — uncallable from outside com.rfo.model	Confirmed as a real bug: Equipment was explicitly private, Insurance implicitly package-private; both would fail to compile the moment UI/service code referenced them	Confirmed
Decided to extract both into standalone top-level enum files	Created Equipment.java and Insurance.java as standalone files, removed the nested declarations	**Accepted — reverses the earlier nesting decision now that cross-package access is actually needed**

16. File persistence design (com.rfo.io)

Prompt (summary)	AI Output	Outcome
Asked whether external libraries could be used for file persistence	Advised against it — the assignment specifically calls for plain File I/O	**Accepted — no external libraries; plain java.io throughout**
Confirmed approach: pipe-delimited text files, one per entity, relationships stored as IDs (not embedded objects), loaded	Outlined the design; flagged several Movement subclasses still needed reconstruction-from-file constructors	Accepted, flagged as follow-up work

in dependency order		
Built LocationFileHandler as the first handler	Provided loadAll()/saveAll(); caught a hidden-file naming issue and recommended checking File.exists() to distinguish “no file yet” from a genuine I/O error	**Accepted — establishes the pattern for the remaining four handlers**
Realized Driver was missing a unique, sequentially-generated driverID	Added driverID with a static auto-incrementing counter, mirroring Movement’s ID pattern	Accepted
Realized DrivenMovement.drivers (List<Driver>) should store just driver IDs, consistent with how staff are stored as String IDs rather than full objects	Changed to driverIDs : List<Integer>; traced the ripple through RentalMovement/WorkshopMovement/CarTransferMovement constructors	**Accepted — applied consistently across all three subclasses**
Raised the password problem — Staff objects never carry a password, but staff.txt needs to store one	Designed StaffFileHandler to work off raw lines internally; authenticate(staffID, password) reads the password column directly without ever constructing a Staff object	**Accepted — password handling fully contained to the io layer**
Worked through adding reconstruction-from-file constructors for every Movement subtype	Added matching constructors to Movement, DrivenMovement, and all five concrete subtypes	**Accepted — full reconstruction support added, unblocking MovementFileHandler**
Designed MovementFileHandler with a type-discriminator column, since one file holds five different concrete Movement subtypes	Built parseLine()/toLine() with type-based dispatch; driver IDs comma-separated, equipment encoded as type:quantity pairs; ID counter primed from the max ID found in the file	Accepted
Hit a compile error: CarTransferMovement’s reconstruction constructor was missing	Diagnosed from the argument-count mismatch in the error message	**Fixed — com.rfo.io package complete**

17. Architecture shift — DataStore in-memory layer

Prompt (summary)	AI Output	Outcome
Asked whether shared getBy/search methods could work across all entity types rather than duplicating per-entity	Explained generics + Predicate<T> as a way to share lookup logic across any entity type via a lambda-based condition	**Accepted — generic Predicate-based lookups on DataStore**
Realized file handlers’ getBy/search/addX methods internally called loadAll() every time, re-reading the whole file from disk on every lookup	Confirmed this was a real design flaw; proposed refactoring all five handlers down to just loadAll()/saveAll(), with search/lookup logic moving to DataStore and the service layer	**Accepted — full io layer refactor**
For StaffFileHandler specifically: saveAll(List<Staff>) had no password data to write, since Staff never holds one — naive implementation would wipe every password on save	Fixed via a read-merge-preserve approach: saveAll() reads existing passwords first, keeps them unchanged; added addNewStaffLine(staff, password) for adding a genuinely new staff member	**Accepted — StaffFileHandler is the one handler with a non-trivial saveAll()**
Asked whether DataStore’s lists should be static, given there will only ever be one instance	Clarified static vs. singleton are different concepts — a single instance created once and passed by reference achieves “only one instance” without static	**Accepted — one DataStore instance, created once, passed by reference everywhere**

18. Service layer built (Auth, Staff, Vehicle, Movement)

Prompt (summary)	AI Output	Outcome
Asked how AuthService should reach StaffFileHandler.authenticate(), since DataStore holds it privately	Presented two options; chose a passthrough method on DataStore	Accepted
Questioned why login() returns a Staff object rather than boolean	Explained this collapses “did it succeed” and “give me the user’s data” into one call	Accepted
Requested VehicleService covering vehicle creation orchestration and defleeting	Built addVehicle() — duplicate check, constructs Vehicle + NewCarMovement together, calls execute(), persists both	Accepted
While drafting MovementService, discovered a real sequencing gap: DrivenMovement.execute() reads odoIn to update the vehicle, but under the two-phase design, odoIn was never actually set anywhere execute() could see it	Fixed at the source: DrivenMovement.closeMovement(int odoIn) now sets odoIn, calls execute() internally, then finalizes status — execute() is no longer called externally	**Accepted — real bug, fixed before it caused incorrect vehicle state**
Asked whether this meant	Confirmed — Java requires overrides to	Accepted

closeMovement() needed to be overloaded in RentalMovement specifically	match parameter lists exactly, so RentalMovement's version with price is an overload, not an override	
Flagged a missing condition: rentals need to check the vehicle is at the staff member's current location before starting	Added as an early guard clause	**Accepted — com.rfo.service layer complete**

19. Smoke testing (Claude Code) — two real bugs found and fixed

Prompt (summary)	AI Output	Outcome
Ran an end-to-end smoke test of the service layer via Claude Code	Bug found: rental cost always $0.00 — Vehicle.rate had no path to ever become non-zero, since the constructor hardcodes 0.0 and nothing ever set it	Confirmed as a real, legitimate gap
—	Fixed: added Vehicle.setRate(double); VehicleService.addVehicle() applies a category-based default rate automatically	Accepted
Rerun surfaced a crash on stale data	Diagnosed: String.split("\	") silently drops trailing empty fields by default; fix is split("\
Further rerun surfaced a NullPointerException tracing back to a null checkOutLoc on a reconstructed movement	Diagnosed the root cause: nothing in the service layer ever called dataStore.saveLocations(), so locations.txt was never written, meaning every restart failed to resolve movement location IDs	Confirmed as a structural gap
Decided to build a proper LocationService	Built addLocation() — duplicate check, adds to DataStore, persists immediately	**Accepted — two genuine bugs (rate stuck at 0, locations never persisted) found and fixed before they could surface during marking**

20. UI layer — Menu base class and navigation

Prompt (summary)	AI Output	Outcome
Designed LoginMenu with a retry limit and one shared Scanner instance passed through every menu	Built with MAX_ATTEMPTS, returns Staff on success or exits after exhausting attempts	Accepted
Designed the overall menu tree with role-based visibility at every level, not just the top level	Confirmed StaffService should own all menu-option filtering logic, keeping UI classes limited to display/input/routing	Accepted
Every sub-menu needs a way to navigate back a level	Recommended an abstract Menu base class handling the shared display/input loop, with subclasses supplying their own title/options/dispatch logic	**Accepted — Menu abstract base class built, used by every concrete menu in the project**
Built MainMenu independently — first draft was missing the required super() call entirely, plus a self-assignment bug on the dataStore field	Flagged both; this same self-assignment pattern (parameter name not matching the field it's meant to assign from) recurred several times across the project	Fixed across several iterations

21. Revenue Movements — checkout, check-in, closing

Prompt (summary)	AI Output	Outcome
Confirmed the two-phase checkIn/close design should be two separate menu actions, matching the real-world scenario	Built checkout() (collects details, calls startRA()), checkin() (stops the clock only), closeRental() (odometer → cost → close)	Accepted
Noticed repeated input-prompting logic (driver IDs, dates, enum selection) duplicated across menu classes	Extracted a shared utility class with generic helpers, including a single generic promptForEnum(Scanner, String, Class<T>) covering every enum-selection prompt	**Accepted — removed duplicated logic across the ui layer**
Found: checkout's location was manually typed by the user rather than derived from their actual current location, meaning the location-match safety check was trivially bypassable	Rebuilt checkout() to derive the checkout location automatically from the logged-in staff member's current location, with validation moved to run immediately after the plate number is entered rather than at the very end of the flow	**Fixed — real security/logic gap, found through manual testing**
Found: checkin() also asked the user to manually type a check-in location	Same fix applied — derived automatically from the staff member's current location	Fixed
Found: the expected check-in date accepted any parseable date, including	Added a check that the date must be after the current time	Fixed

ones years in the past		
Found: AdminMenu's Change Location prompt expected a numeric list index, but the displayed list showed location codes, causing every attempt typing a code to silently fail	Rebuilt to accept the location code directly, and to display the staff member's current location alongside the choices	Fixed
Requested login and staffID lookups be case-insensitive	Applied .toUpperCase() normalization at the point of input across login and staff-lookup prompts	Accepted

22. Non-Revenue Movements, Fleet, Reports

Prompt (summary)	AI Output	Outcome
Built NonRevenueMovementsMenu (New Vehicle, Car Transfer, Workshop, Defleet), mapping to existing VehicleService/MovementService methods	Provided full class using the shared prompt utilities	Accepted
Requested Query Available Vehicles support two sort modes — general (by category, then odometer, then plate) and specific (one category, by odometer then plate)	Built using chained Comparator methods, relying on VehicleCategory's natural enum declaration order for the category sort	Accepted
Specced all four Reports screens, including exact sort orders and column formats	Built all four using dataStore.search() with filters and Comparator chains	Accepted
Wrote a status filter comparing MovementStatus against a string literal ("OPEN") instead of the enum constant	Caught this as a real, silent bug — comparing an enum to a string always returns false, meaning the filter would silently match nothing with no error	**Fixed — a recurring class of bug worth noting: enum values must be compared to enum constants, not strings**
Clarified Out on Rental Report should be scoped to the logged-in staff member's own current location	Rebuilt with a location match condition	Accepted
Clarified New Vehicles report should query Vehicle directly (odometer ≤ 30 and status AVAILABLE) rather than querying movement history	Rewrote as a direct Vehicle filter	**Accepted — Fleet and Reports menus complete**

23. Location Manager — staff and location management

Prompt (summary)	AI Output	Outcome
Decided Manage Staff should support viewing, adding, and granting location access — closing out a feature deferred all the way back during initial design	Built StaffService.grantLocationAccess(), checking for existing access before adding	Accepted
Decided Manage Locations should mirror this with add/close functionality; "removing" a staff member should revoke all their location access rather than deleting the record (staffID stays referenced by historical movements); "removing" a location should be a close, not a delete, since old movement records still reference it	Location gained an `active` boolean field; closing a location bulk-relocates any vehicles still there via a real CarTransferMovement (using a placeholder driver ID, matching a real office workaround) rather than editing their location silently, then marks the location inactive rather than deleting it	**Accepted — full feature built across the model, service, and ui layers**
Caught a bug during this build: closeLocation() redeclared DataStore as a method parameter even though it was already a field on the class, silently shadowing the field	Flagged and fixed	Fixed
Ran a full-project audit	Found two real structural gaps: MainMenu still hardcoded "coming soon" for three fully-built menus instead of routing to them; LocationManagerMenu still contained old logic instead of delegating to its new sub-menus, making the new staff/location management features unreachable. Also found closeLocation() started vehicle transfers but never completed them, leaving vehicles stranded mid-move	All three fixed and reverified

24. RFOApp — entry point

Prompt (summary)	AI Output	Outcome
Designed the entry point: one Scanner and one DataStore created exactly once, login, then hand off to the main session	First draft had MainMenu constructed but never actually run (`.run()` was missing), and a missing `continue` meant a failed login could still fall through to starting a session with no staff member	Caught and fixed across two iterations
Confirmed logout should end the whole program rather than loop back to a new login screen	Simplified RFOApp from a loop to a single straight-line flow: login once, run one session, exit	**Accepted — application complete and runnable end-to-end**

25. Role-based menu trimming

Prompt (summary)	AI Output	Outcome
Specified the full role-visibility breakdown for every menu level, not just the top level (previously every sub-menu showed the same full option set to every role)	Built role-filtered option methods on StaffService for every menu level, updated each menu class's getOptions() to use them	Accepted
Flagged a resulting gap: a role with zero permitted options in a category could still select that category and land in an empty sub-menu with nothing but an exit option	Fixed by having each parent menu check whether a category has any visible options before showing it at all	**Accepted — role-based trimming complete and consistent at every level of the menu tree**

This log covers the design and code decisions made throughout the development of RFO.

Appendix B: One-Page Reflective Summary on AI Use

Appendix B — Reflective Summary on AI Use

I used Claude (Anthropic) throughout this project as a design sounding board and code reviewer, not as a code generator. For almost every class, I wrote the first version myself, then shared it for review, where the AI's job was to catch mistakes and question decisions, similar to peer review work, not to produce working code for me to paste in. The handful of cases where I did have it generate code directly were ones where a pattern had already been established across earlier classes or methods, and what remained was applying that same semi-repeated structure to the rest (the file handlers' reconstruction constructors once the pattern was set on the first one, or the menu classes once the shared base class and navigation pattern were in place). AI never had direct access to my codebase. Every change went through me typing it into NetBeans myself, and even generated classes were only pasted in after close reading and reviewing.

The most useful pattern across the project was being pushed to defend my own design decisions with real detail rather than abstract reasoning. Early on, I proposed a Vehicle class hierarchy with a subclass per category (Compact, Intermediate, Standard, and so on). The AI's first response was cautious but open to it. It was only once I gave a concrete example, that the same category, Intermediate, actually contains vehicles with completely different GPS and fuel combinations in real fleets, that it became clear the variation was at the level of an individual vehicle, not the category. That single example was enough to collapse what would have been an unnecessary five-class hierarchy into one enum. The same thing happened with Staff: I had planned subclasses per role, but once I explained the only real difference was permissions, not data, the design correctly moved to a StaffRole enum instead. In both cases the AI's first suggestion was reasonable given what I'd told it, and it changed once I gave it the domain knowledge it didn't have. That discussion dynamic was exactly what I needed from an AI tool. It was a way to pressure-test my thinking and arrive at a genuinely concrete design.

The other place AI usage was genuinely valuable was in catching a specific, recurring class of bug I would likely have missed reading my own code back: constructor parameters that don't quite match the field they're meant to assign to, silently self-assigning a field to itself instead. This happened several separate times across the project, in different classes, and each time it was caught immediately during review because it's an easy pattern to check for mechanically, even though it's easy to miss visually. A second recurring bug type was comparing an enum value against a plain string instead of the enum constant itself. This compiles fine and never throws an error, it just silently matches nothing, which is a genuinely hard bug to notice without someone specifically looking for it.

Beyond design and review, I used Claude Code separately to run smoke tests against the finished service and UI layers, treating it as an independent tester rather than part of the build process. This surfaced two real bugs that had nothing to do with typos or syntax: a vehicle's rental rate had no path to ever become non-zero, since nothing in the service layer ever set it, and locations were never actually being written to disk, meaning the whole application would silently lose location data the moment it restarted. Both were the kind of structural gap that's easy to miss when you're the one who wrote the code, since I already knew what I meant a given method to do.

AI also made it easier to organize and check the AI usage log itself, and to write this reflective summary. English is my second language, and although I am fluent, using AI here helped me express things more clearly and accurately.

Overall, AI use on this project was concentrated on architecture discussions and code review rather than code generation, and the value came mostly from being asked to justify decisions and having a second pass over code looking specifically for the kinds of mistakes that don't show up as compiler errors.